Forecasting Electricity Demand

A worked case study in time-series forecasting, with a real generation-mix leakage trap

Your Name

2026-10-16

Table of contents

1 Introduction **2**

2 Data **2**
- 2.1 Three real data-quality issues — and they are not the same issue 3
  - 2.1.1 A third issue: fabricated readings in 2025-2026 3
- 2.2 Exploring the series and its generation mix 8

3 Statistical rigor before modeling **11**
- 3.1 Is this series stationary? 11
- 3.2 Autocorrelation, used to justify lag choices empirically 11

4 Feature engineering **12**
- 4.1 The leakage trap in the generation-mix columns 12
- 4.2 A leakage-safe way to reuse a seasonal-decomposition idea 13

5 Methodology **14**
- 5.1 Why the train/test split is chronological, not shuffled 14
- 5.2 Two baselines, measured before trusting any model 14
- 5.3 Model comparison, via TimeSeriesSplit on the training period 15

6 Final chronological holdout **16**

7 Forecasting 24 hours ahead: where the model earns its keep **18**

8 Model interpretability **20**

9 Discussion: what this project can't do, and what a Vietnam version would need **21**

10 Conclusion 22

Interactive application

Open the Streamlit app

1 Introduction

I built this project to predict hourly electricity demand (`Consumption`, megawatts) for Romania's national grid, using real, published data from 2019 onward, together with its generation mix (nuclear, wind, hydro, oil & gas, coal, solar, biomass). I picked demand forecasting as a deliberately different kind of project for this portfolio: every other project treats each row as independent, while this one is a **time series**, where the order of the rows *is* the problem and a shuffled train/test split would quietly leak.

I originally built this project on US grid data (PJM Interconnection), then switched to this Romanian dataset specifically because the generation-mix breakdown is genuinely useful, interesting data the US version didn't have — it lets me ask whether recent generation mix carries any forecasting signal, and it comes with its own real energy-transition story.

One thing worth flagging upfront: **this dataset is live and continuously updated** — checked directly, an earlier download had 46,002 rows through April 2024, the version used here has 62,810 rows through March 2026, with older rows unchanged. Every number below reflects one snapshot, not a fixed, versioned file.

I still couldn't find a real, public, hourly demand dataset for Vietnam's own grid. Romania is a different climate and market structure, but grid-scale demand forecasting is the same *kind* of problem regardless of country, and Romania's growing renewable share raises questions increasingly relevant to Vietnam's own energy transition too.

2 Data

```
from energy_demand_forecasting import config, data

df = data.load_raw()
print(f"{len(df)} rows, {df[config.DATETIME_COL].min()} to {df[config.DATETIME_COL].max()}")
```

```
62810 rows, 2019-01-01 00:00:00 to 2026-03-14 23:00:00
```

2.1 Three real data-quality issues — and they are not the same issue

```
dupes = df[df[config.DATETIME_COL].duplicated(keep=False)]
full_range = pd.date_range(df[config.DATETIME_COL].min(), df[config.DATETIME_COL].max(), freq
missing = full_range.difference(df[config.DATETIME_COL])
n_extra = df[config.DATETIME_COL].duplicated().sum()
print(f"{dupes[config.DATETIME_COL].nunique()} timestamps appear more than once "
      f"({len(dupes)} rows, i.e. {n_extra} extra copies); {len(missing)} missing timestamps")

gaps = data.gap_lengths(df)
print(f"{len(gaps)} separate gaps, longest: {gaps.max()} hours")
```

```
7 timestamps appear more than once (18 rows, i.e. 11 extra copies); 321 missing timestamps
78 separate gaps, longest: 83 hours
```

Every duplicate falls on an October date at **3am** local time — the same Daylight Saving Time fall-back pattern as any DST-observing country, just an hour later on the clock than the US version of this project found, since Romania is in the Eastern European time zone. 2019-2023 show exactly 1 missing hour per year (the spring-forward gap). **2024-2025 are a completely different story**: a single gap of **83 consecutive missing hours**, consistent with a real reporting/scraping outage in whatever live feed this dataset's maintainer pulls from — not a calendar quirk. I fixed both the same way (average duplicates, linearly interpolate gaps against a strictly regular hourly index), but want to be upfront that interpolating an 83-hour gap is a much rougher approximation than the isolated 1-hour DST gaps — there's no way to recover 83 hours of real demand swings from a straight line between two endpoints.

2.1.1 A third issue: fabricated readings in 2025-2026

The largest `Consumption` values in the raw file are not cold snaps. They sit in the middle of the night, climb hour after hour to 10,000-12,000 MW (the raw maximum is 11,986 MW at 06:00 on 27 November 2025), and then fall back by thousands of MW within a single hour. Checking every column at those hours shows what is going on: each row is an **exact scaled-up copy of the row before it** — `Production`, `Nuclear`, `Hydroelectric`, `Oil and Gas`, `Coal` and `Consumption` all change by the same factor, hour after hour. Real hours never do that: nuclear output is nearly constant while demand moves, and here it climbs to about 2,080 MW, well beyond what Romania's only nuclear plant (Cernavoda, two ~700 MW units) can produce. It looks like a fault in the live feed that fills hours by rescaling the last real reading.

`data.flag_implausible_readings` (called inside `clean_frame`) removes these with a deliberately conservative, documented rule (constants in `config.py`):

1. **Trigger** — an hour is clearly impossible if `Consumption` is more than 1.5x the median of the same clock hour over the surrounding ±7 days, *or* `Nuclear` exceeds 1,500 MW. Neither ever happens in 2019-2024 (the highest ratio there is 1.46, the highest nuclear reading 1,457 MW). Unusually *low* hours are never flagged: real holidays fall below half the usual level.
2. **Extent** — the flag spreads over the contiguous run of hours that are exact proportional copies of the hour before (all six columns above changing by the same factor, within 1%), since the whole run is produced by the same fault. The genuine hour being copied is kept.

Flagged hours are set to missing in every column and interpolated like the gaps above.

```
flagged = data.implausible_readings(df)
stretch_id = (flagged.index.to_series().diff() != pd.Timedelta(hours=1)).cumsum()
stretches = flagged.groupby(stretch_id.values).agg(
    start=("Consumption", lambda s: s.index.min()),
    end=("Consumption", lambda s: s.index.max()),
    hours=("Consumption", "size"),
    max_consumption_MW=("Consumption", "max"),
    max_ratio_to_usual=("ratio", "max"),
    max_nuclear_MW=("Nuclear", "max"),
).reset_index(drop=True)
print(f"{len(flagged)} hours flagged, in {len(stretches)} separate stretches, "
      f"all between {flagged.index.min():%Y-%m-%d} and {flagged.index.max():%Y-%m-%d}")
stretches.round(2)
```

```
123 hours flagged, in 16 separate stretches, all between 2025-03-26 and 2026-
02-11
```

	start	end	hours	max_consumption_MW	max_ratio_to_usual	ma
0	2025-03-26 23:00:00	2025-03-27 07:00:00	9	8735.0	1.48	185
1	2025-04-22 08:00:00	2025-04-22 09:00:00	2	5607.0	1.01	150
2	2025-04-24 08:00:00	2025-04-24 13:00:00	6	6561.0	1.43	159
3	2025-04-25 17:00:00	2025-04-25 22:00:00	6	5743.0	0.97	154
4	2025-04-29 17:00:00	2025-04-29 18:00:00	2	5479.0	0.96	152
5	2025-05-09 07:00:00	2025-05-09 08:00:00	2	6137.0	1.05	153
6	2025-05-25 19:00:00	2025-05-26 05:00:00	11	7631.0	1.51	910
7	2025-08-07 20:00:00	2025-08-08 07:00:00	12	10112.0	1.79	180
8	2025-09-14 23:00:00	2025-09-15 05:00:00	7	6441.0	1.28	159
9	2025-10-02 17:00:00	2025-10-03 05:00:00	13	10179.0	1.93	192
10	2025-10-04 14:00:00	2025-10-04 19:00:00	6	7113.0	1.09	159
11	2025-10-10 20:00:00	2025-10-11 06:00:00	11	10914.0	1.94	200

	start	end	hours	max_consumption_MW	max_ratio_to_usual	ma
12	2025-11-26 17:00:00	2025-11-27 06:00:00	14	11986.0	2.02	208
13	2026-01-11 21:00:00	2026-01-12 05:00:00	9	9789.0	1.56	173
14	2026-01-24 02:00:00	2026-01-24 10:00:00	9	8062.0	1.09	172
15	2026-02-11 19:00:00	2026-02-11 22:00:00	4	9460.0	1.33	151

The six stretches that reach 1.5x the usual level contain every one of the dataset's 10,000-12,000 MW "peaks"; the others are milder copies of the same fault, caught because their nuclear reading is physically impossible. The complete list of flagged hours:

All flagged hours (raw values, with the same-hour median they were judged against)

DateTime	Consumption	Production	Nuclear	same_hour_median	ratio
2025-03-26 23:00:00	6713.0	6226.0	1426.0	6067.0	1.11
2025-03-27 00:00:00	6780.0	6288.0	1440.0	5734.5	1.18
2025-03-27 01:00:00	7254.0	6728.0	1540.0	5528.0	1.31
2025-03-27 02:00:00	7544.0	6997.0	1601.0	5434.0	1.39
2025-03-27 03:00:00	7770.0	7206.0	1649.0	5395.0	1.44
2025-03-27 04:00:00	8080.0	7494.0	1714.0	5458.0	1.48
2025-03-27 05:00:00	8241.0	7643.0	1748.0	5582.5	1.48
2025-03-27 06:00:00	8241.0	7643.0	1748.0	5945.0	1.39
2025-03-27 07:00:00	8735.0	8101.0	1852.0	6342.5	1.38
2025-04-22 08:00:00	5444.0	3627.0	1463.0	5582.5	0.98
2025-04-22 09:00:00	5607.0	3735.0	1506.0	5530.0	1.01
2025-04-24 08:00:00	5834.0	4433.0	1422.0	5309.5	1.10
2025-04-24 09:00:00	5834.0	4433.0	1422.0	5261.0	1.11
2025-04-24 10:00:00	6067.0	4610.0	1478.0	4974.5	1.22
2025-04-24 11:00:00	6249.0	4748.0	1522.0	4689.0	1.33
2025-04-24 12:00:00	6561.0	4985.0	1598.0	4586.5	1.43
2025-04-24 13:00:00	6561.0	4985.0	1598.0	4588.5	1.43
2025-04-25 17:00:00	5059.0	5146.0	1365.0	5239.5	0.97
2025-04-25 18:00:00	5059.0	5146.0	1365.0	5567.5	0.91
2025-04-25 19:00:00	5311.0	5403.0	1433.0	5932.5	0.90
2025-04-25 20:00:00	5311.0	5403.0	1433.0	6080.0	0.87
2025-04-25 21:00:00	5470.0	5565.0	1475.0	6213.0	0.88
2025-04-25 22:00:00	5743.0	5843.0	1548.0	5999.0	0.96
2025-04-29 17:00:00	5169.0	4731.0	1438.0	5401.5	0.96
2025-04-29 18:00:00	5479.0	5014.0	1524.0	5784.0	0.95
2025-05-09 07:00:00	5845.0	5856.0	1464.0	5827.5	1.00
2025-05-09 08:00:00	6137.0	6148.0	1537.0	5856.5	1.05

2025-05-25 19:00:00	5530.0	4432.0	662.0	6480.0	0.85
2025-05-25 20:00:00	5640.0	4520.0	675.0	6748.5	0.84
2025-05-25 21:00:00	5922.0	4746.0	708.0	6860.0	0.86
2025-05-25 22:00:00	6218.0	4983.0	743.0	6340.5	0.98
2025-05-25 23:00:00	6653.0	5331.0	795.0	5805.0	1.15
2025-05-26 00:00:00	6919.0	5544.0	826.0	5427.0	1.27
2025-05-26 01:00:00	6988.0	5599.0	834.0	5198.0	1.34
2025-05-26 02:00:00	7197.0	5766.0	859.0	5056.0	1.42
2025-05-26 03:00:00	7556.0	6054.0	901.0	5060.5	1.49
2025-05-26 04:00:00	7631.0	6114.0	910.0	5046.0	1.51
2025-05-26 05:00:00	7631.0	6114.0	910.0	5110.5	1.49
2025-08-07 20:00:00	7119.0	6113.0	1317.0	7191.5	0.99
2025-08-07 21:00:00	7546.0	6479.0	1396.0	7214.5	1.05
2025-08-07 22:00:00	7546.0	6479.0	1396.0	6767.5	1.12
2025-08-07 23:00:00	7696.0	6608.0	1423.0	6182.0	1.24
2025-08-08 00:00:00	8080.0	6938.0	1494.0	5649.0	1.43
2025-08-08 01:00:00	8564.0	7354.0	1583.0	5395.5	1.59
2025-08-08 02:00:00	8906.0	7648.0	1646.0	5233.5	1.70
2025-08-08 03:00:00	9084.0	7800.0	1678.0	5144.0	1.77
2025-08-08 04:00:00	9174.0	7878.0	1694.0	5112.5	1.79
2025-08-08 05:00:00	9174.0	7878.0	1694.0	5136.0	1.79
2025-08-08 06:00:00	9540.0	8193.0	1761.0	5403.5	1.77
2025-08-08 07:00:00	10112.0	8684.0	1866.0	5838.5	1.73
2025-09-14 23:00:00	5457.0	5213.0	1349.0	5751.0	0.95
2025-09-15 00:00:00	5511.0	5265.0	1362.0	5349.0	1.03
2025-09-15 01:00:00	5566.0	5317.0	1375.0	5154.0	1.08
2025-09-15 02:00:00	5732.0	5476.0	1416.0	4993.5	1.15
2025-09-15 03:00:00	6133.0	5859.0	1515.0	4931.0	1.24
2025-09-15 04:00:00	6378.0	6093.0	1575.0	4974.5	1.28
2025-09-15 05:00:00	6441.0	6153.0	1590.0	5163.0	1.25
2025-10-02 17:00:00	7034.0	6543.0	1331.0	6630.5	1.06
2025-10-02 18:00:00	7385.0	6870.0	1397.0	6869.0	1.08
2025-10-02 19:00:00	7532.0	7007.0	1424.0	7251.5	1.04
2025-10-02 20:00:00	8059.0	7497.0	1523.0	7343.5	1.10
2025-10-02 21:00:00	8461.0	7871.0	1599.0	6965.0	1.21
2025-10-02 22:00:00	9053.0	8421.0	1710.0	6287.5	1.44
2025-10-02 23:00:00	9053.0	8421.0	1710.0	5801.0	1.56
2025-10-03 00:00:00	9053.0	8421.0	1710.0	5453.5	1.66
2025-10-03 01:00:00	9053.0	8421.0	1710.0	5288.5	1.71
2025-10-03 02:00:00	9053.0	8421.0	1710.0	5155.0	1.76
2025-10-03 03:00:00	9234.0	8589.0	1744.0	5099.0	1.81

2025-10-03 04:00:00	9603.0	8932.0	1813.0	5141.5	1.87
2025-10-03 05:00:00	10179.0	9467.0	1921.0	5270.0	1.93
2025-10-04 14:00:00	6329.0	5036.0	1423.0	5910.0	1.07
2025-10-04 15:00:00	6518.0	5187.0	1465.0	5988.5	1.09
2025-10-04 16:00:00	6518.0	5187.0	1465.0	6329.0	1.03
2025-10-04 17:00:00	6974.0	5550.0	1567.0	6676.5	1.04
2025-10-04 18:00:00	6974.0	5550.0	1567.0	6968.5	1.00
2025-10-04 19:00:00	7113.0	5661.0	1598.0	7367.0	0.97
2025-10-10 20:00:00	7546.0	6436.0	1389.0	7480.5	1.01
2025-10-10 21:00:00	7546.0	6436.0	1389.0	7118.0	1.06
2025-10-10 22:00:00	8074.0	6886.0	1486.0	6422.0	1.26
2025-10-10 23:00:00	8396.0	7161.0	1545.0	5972.5	1.41
2025-10-11 00:00:00	8815.0	7519.0	1622.0	5623.0	1.57
2025-10-11 01:00:00	9343.0	7970.0	1719.0	5429.5	1.72
2025-10-11 02:00:00	9623.0	8209.0	1770.0	5310.5	1.81
2025-10-11 03:00:00	10200.0	8701.0	1876.0	5265.0	1.94
2025-10-11 04:00:00	10200.0	8701.0	1876.0	5297.0	1.93
2025-10-11 05:00:00	10200.0	8701.0	1876.0	5475.5	1.86
2025-10-11 06:00:00	10914.0	9310.0	2007.0	6015.5	1.81
2025-11-26 17:00:00	8113.0	8243.0	1413.0	8142.0	1.00
2025-11-26 18:00:00	8194.0	8325.0	1427.0	8107.5	1.01
2025-11-26 19:00:00	8685.0	8824.0	1512.0	7985.5	1.09
2025-11-26 20:00:00	8945.0	9088.0	1557.0	7743.5	1.16
2025-11-26 21:00:00	8945.0	9088.0	1557.0	7224.5	1.24
2025-11-26 22:00:00	9571.0	9724.0	1665.0	6676.0	1.43
2025-11-26 23:00:00	9571.0	9724.0	1665.0	6195.5	1.54
2025-11-27 00:00:00	10240.0	10404.0	1781.0	5851.0	1.75
2025-11-27 01:00:00	10444.0	10612.0	1816.0	5690.0	1.84
2025-11-27 02:00:00	10861.0	11036.0	1888.0	5589.5	1.94
2025-11-27 03:00:00	11078.0	11256.0	1925.0	5566.5	1.99
2025-11-27 04:00:00	11299.0	11481.0	1963.0	5650.5	2.00
2025-11-27 05:00:00	11637.0	11825.0	2021.0	5774.0	2.02
2025-11-27 06:00:00	11986.0	12179.0	2081.0	6449.0	1.86
2026-01-11 21:00:00	7894.0	6836.0	1405.0	7718.0	1.02
2026-01-11 22:00:00	7894.0	6836.0	1405.0	7183.5	1.10
2026-01-11 23:00:00	7972.0	6904.0	1419.0	6670.0	1.20
2026-01-12 00:00:00	8290.0	7180.0	1475.0	6340.0	1.31
2026-01-12 01:00:00	8787.0	7610.0	1563.0	6268.5	1.40
2026-01-12 02:00:00	8962.0	7762.0	1594.0	6103.0	1.47
2026-01-12 03:00:00	9141.0	7917.0	1625.0	6075.0	1.50
2026-01-12 04:00:00	9323.0	8075.0	1657.0	6054.5	1.54

2026-01-12 05:00:00	9789.0	8478.0	1739.0	6272.0	1.56
2026-01-24 02:00:00	6316.0	5455.0	1352.0	6036.5	1.05
2026-01-24 03:00:00	6316.0	5455.0	1352.0	6040.5	1.05
2026-01-24 04:00:00	6631.0	5727.0	1419.0	6078.0	1.09
2026-01-24 05:00:00	6896.0	5956.0	1475.0	6323.0	1.09
2026-01-24 06:00:00	6896.0	5956.0	1475.0	6942.0	0.99
2026-01-24 07:00:00	7240.0	6253.0	1548.0	7732.0	0.94
2026-01-24 08:00:00	7457.0	6440.0	1594.0	8243.0	0.90
2026-01-24 09:00:00	7606.0	6568.0	1625.0	8376.5	0.91
2026-01-24 10:00:00	8062.0	6962.0	1722.0	8231.5	0.98
2026-02-11 19:00:00	8334.0	7994.0	1338.0	8295.0	1.00
2026-02-11 20:00:00	8500.0	8153.0	1364.0	8111.5	1.05
2026-02-11 21:00:00	8925.0	8560.0	1432.0	7696.5	1.16
2026-02-11 22:00:00	9460.0	9073.0	1517.0	7139.5	1.33

This is not a perfect fix. Interpolating a 9-14 hour overnight stretch with a straight line misses the real night-time dip, and milder copies that trip neither trigger (as well as some older stretches where the feed simply repeated one reading for several hours) are left as they are; the rule only removes values that are clearly impossible.

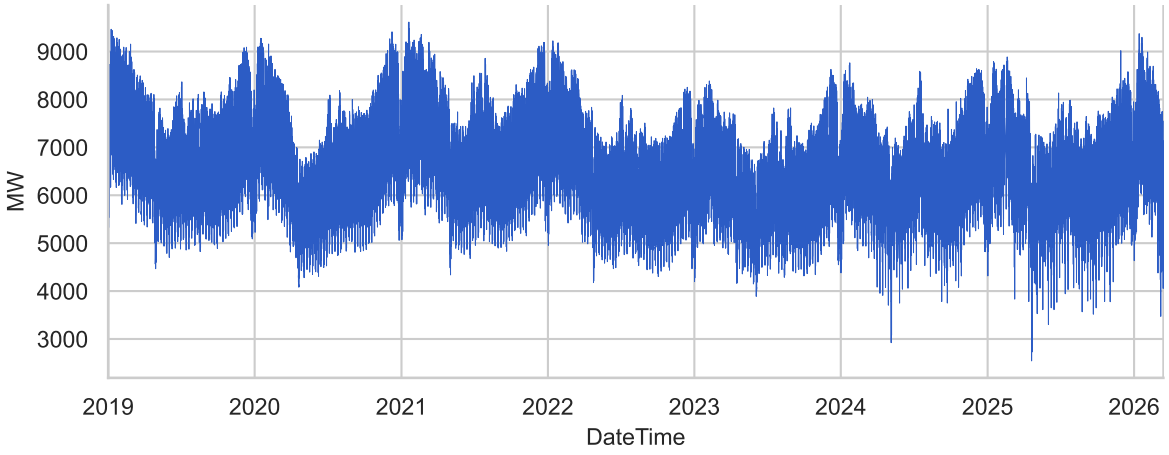
```
frame = data.load_clean_frame()
print(f"{len(frame)} hours, strictly regular: {(frame.index.to_series().diff().dropna() == pd
print(f"Peak demand after cleaning: {frame[config.TARGET_COL].max():,.0f} MW "
      f"at {frame[config.TARGET_COL].idxmax()} (raw maximum: {df[config.TARGET_COL].max():,.0
```

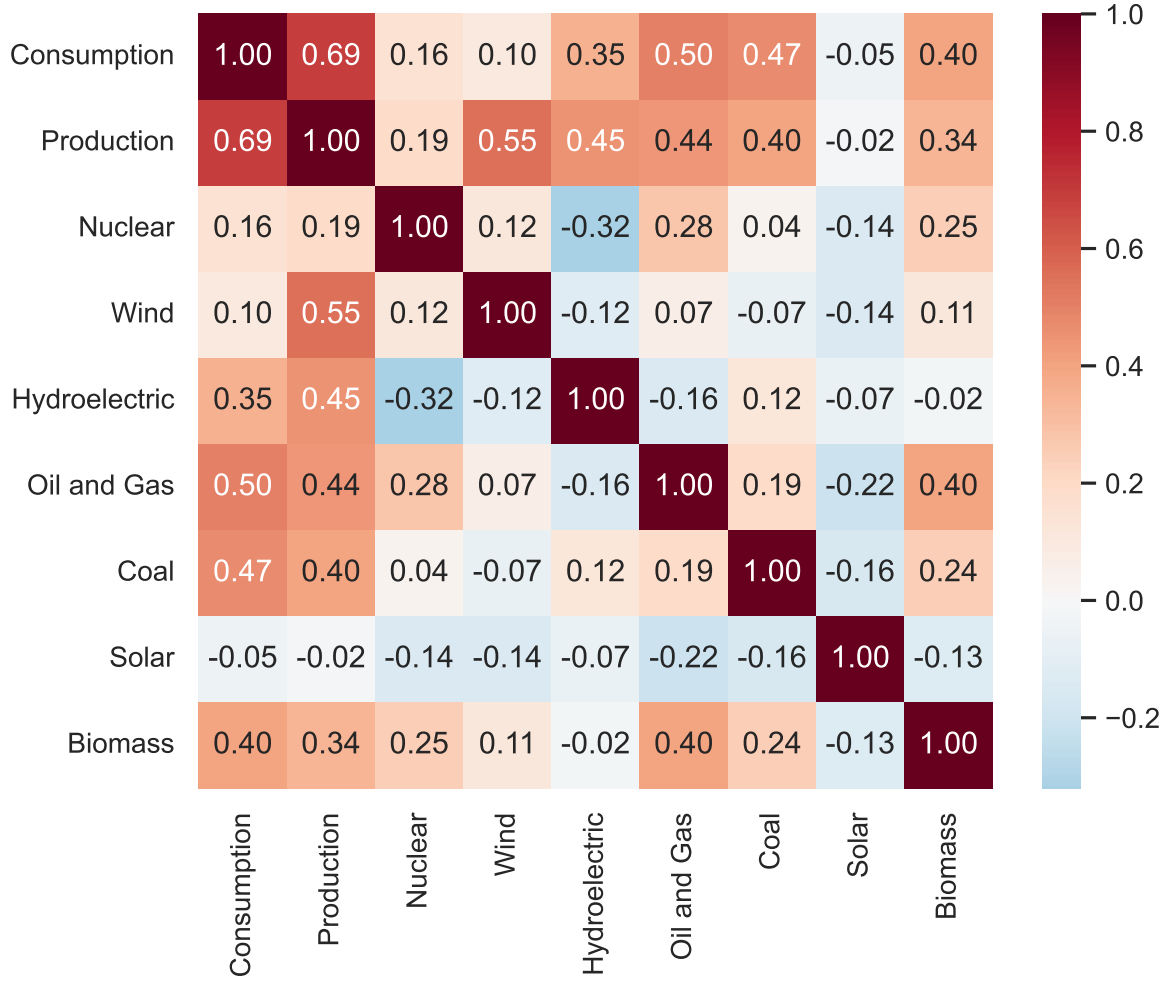
```
63120 hours, strictly regular: True
Peak demand after cleaning: 9,615 MW at 2021-01-19 10:00:00 (raw maximum: 11,986 MW)
```

2.2 Exploring the series and its generation mix

```
fig, ax = plt.subplots(figsize=(9, 3.2))
frame[config.TARGET_COL].plot(ax=ax, linewidth=0.4, color=PALETTE[0])
ax.set_ylabel("MW")
plt.show()

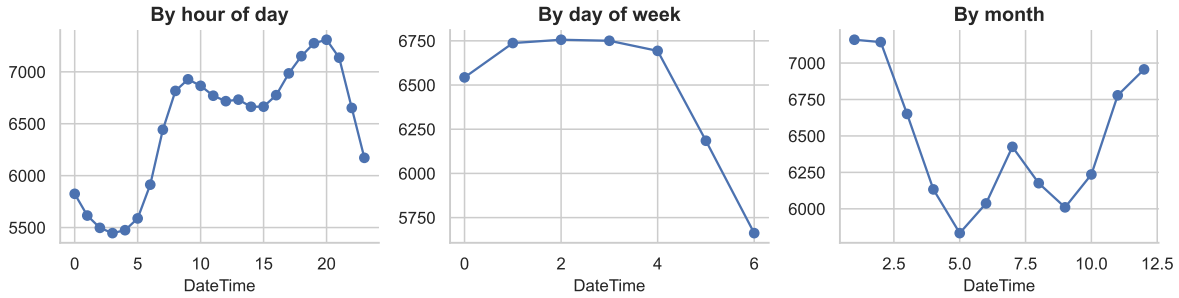
corr = frame.corr()
fig, ax = plt.subplots(figsize=(7, 5.5))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0, ax=ax)
plt.show()
```


9000
8000
7000
6000
5000
4000
3000
MW
2019
2020
2021
2022
2023
2024
2025
2026
DateTime


Consumption 1.00 0.69 0.16 0.10 0.35 0.50 0.47 -0.05 0.40
Production 0.69 1.00 0.19 0.55 0.45 0.44 0.40 -0.02 0.34
Nuclear 0.16 0.19 1.00 0.12 -0.32 0.28 0.04 -0.14 0.25
Wind 0.10 0.55 0.12 1.00 -0.12 0.07 -0.07 -0.14 0.11
Hydroelectric 0.35 0.45 -0.32 -0.12 1.00 -0.16 0.12 -0.07 -0.02
Oil and Gas 0.50 0.44 0.28 0.07 -0.16 1.00 0.19 -0.22 0.40
Coal 0.47 0.40 0.04 -0.07 0.12 0.19 1.00 -0.16 0.24
Solar -0.05 -0.02 -0.14 -0.14 -0.07 -0.22 -0.16 1.00 -0.13
Biomass 0.40 0.34 0.25 0.11 -0.02 0.40 0.24 -0.13 1.00
Consumption
Production
Nuclear
Wind
Hydroelectric
Oil and Gas
Coal
Solar
Biomass
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

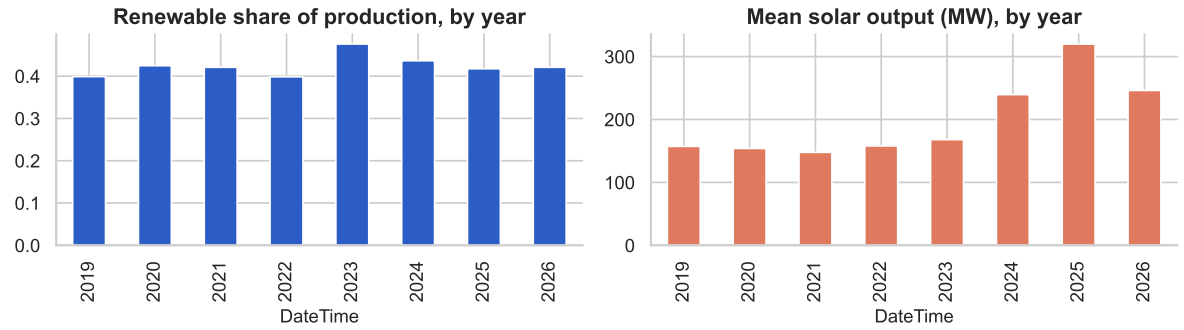
`Consumption` correlates most with `Production` (0.69, as expected) and the two dispatchable fossil sources, `Oil and Gas` (0.50) and `Coal` (0.47) — the grid operator leans on fossil generation specifically to follow demand swings. `Solar` is essentially uncorrelated with demand (-0.05), which is exactly what I'd expect: solar output follows daylight and weather, not consumption.

```
fig, axes = plt.subplots(1, 3, figsize=(11, 3.0))
frame.groupby(frame.index.hour)[config.TARGET_COL].mean().plot(ax=axes[0], marker="o", title=
frame.groupby(frame.index.dayofweek)[config.TARGET_COL].mean().plot(ax=axes[1], marker="o", t
frame.groupby(frame.index.month)[config.TARGET_COL].mean().plot(ax=axes[2], marker="o", titl
plt.tight_layout()
plt.show()
```



A single evening peak (~8-9pm, ~7,300 MW) and overnight trough (~3-4am, ~5,450 MW) — a cleaner single-peak daily shape than this project's original US version, which had a secondary morning bump. The monthly pattern is **winter-dominant, not bimodal**: demand peaks in January/February and troughs in May, with much less of a summer cooling bump than the US version showed — consistent with Romania's historically lower AC penetration.

```
renewable_share = frame[config.RENEWABLE_COLS].sum(axis=1) / frame["Production"]
fig, axes = plt.subplots(1, 2, figsize=(10, 3.0))
renewable_share.groupby(frame.index.year).mean().plot.bar(ax=axes[0], title="Renewable share
frame.groupby(frame.index.year)["Solar"].mean().plot.bar(ax=axes[1], title="Mean solar output
plt.tight_layout()
plt.show()
```



The clearest "genuinely useful and interesting" finding from having the generation-mix breakdown at all: renewable share has held around 40-48% of production every year since 2019, but **mean solar output roughly doubled between 2023 (~168 MW) and 2025 (~320 MW)** — a real, checkable signature of Romania's recent solar buildout.

3 Statistical rigor before modeling

3.1 Is this series stationary?

```
from energy_demand_forecasting import model

adf_result = model.stationarity_test(frame[config.TARGET_COL])
print(f"ADF statistic: {adf_result['statistic']:.2f}, p-value: {adf_result['p_value']:.4g}")
```

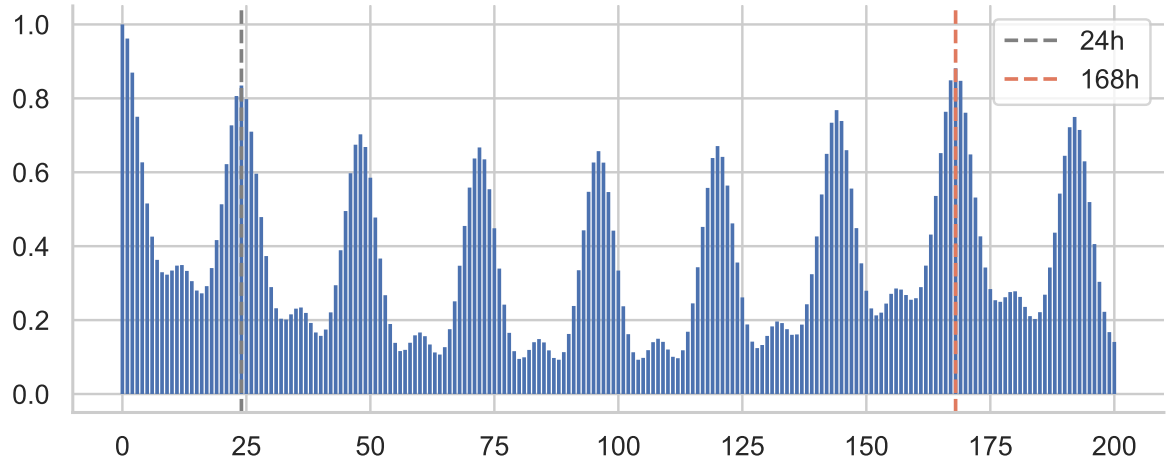
```
ADF statistic: -14.06, p-value: 3.045e-26
```

Formally stationary (no unit root) — which looked wrong to me at first given the obvious daily/monthly patterns above. The resolution: ADF specifically tests for a *stochastic trend*, not for seasonality. A strong, bounded, repeating pattern always reverts toward its typical shape rather than wandering further from it, so it doesn't have a unit root even though it clearly isn't "flat."

3.2 Autocorrelation, used to justify lag choices empirically

```
acf_values = model.autocorrelation(frame[config.TARGET_COL], n_lags=200)
fig, ax = plt.subplots(figsize=(8, 3.0))
ax.stem(range(len(acf_values)), acf_values, markerfmt=" ", basefmt=" ")
```

```
ax.axvline(24, linestyle="--", color="gray", label="24h")
ax.axvline(168, linestyle="--", color=PALETTE[3], label="168h")
ax.legend()
plt.show()
for lag in [1, 24, 48, 168]:
    print(f"ACF at lag {lag}h: {acf_values[lag]:.4f}")
```




```
ACF at lag 1h: 0.9618
ACF at lag 24h: 0.8347
ACF at lag 48h: 0.7026
ACF at lag 168h: 0.8813
```

Autocorrelation is genuinely high at both 24h (0.83) and 168h (0.88 — if anything slightly stronger), with a noticeably weaker mid-point at 48h (0.70) — empirical justification for these two specific lags, rather than picking them by convention.

4 Feature engineering

4.1 The leakage trap in the generation-mix columns

`Production` and its 7 source breakdowns are recorded at the **same hour** as `Consumption`, not before it. Using them directly would be circular for a real forecasting task, since generation is dispatched in response to demand, not fixed in advance. Every generation-mix feature is lagged by 24 hours.

```
from energy_demand_forecasting import features

seasonal_lookup = features.fit_seasonal_lookup(frame[config.TARGET_COL].iloc[: int(len(frame)
X, y = features.split_features_target(frame, seasonal_lookup=seasonal_lookup)
X[["lag_24h", "lag_168h", "Solar_lag_24h", "net_export_lag_24h", "renewable_share_lag_24h"]]
```

DateTime	lag_24h	lag_168h	Solar_lag_24h	net_export_lag_24h	renewable_share_lag_24
2026-03-14 21:00:00	7041.0	6402.0	0.0	-185.0	0.441219
2026-03-14 22:00:00	6426.0	6234.0	0.0	115.0	0.415839
2026-03-14 23:00:00	6019.0	5777.0	0.0	348.0	0.394220

4.2 A leakage-safe way to reuse a seasonal-decomposition idea

A natural next step is decomposing the series into trend/seasonal/ residual components and adding them as features. Doing this the most direct way — fitting `statsmodels`' decomposition on the whole series before splitting — leaks: the trend component is a **centered** moving average using points both before *and after* each timestamp, so fitting it on the full series means training features are quietly built with future test-period information. `fit_seasonal_lookup` fits the decomposition **only on the training split** and reuses just the strictly periodic seasonal component (24 fixed hour-of-day offsets, safe to reapply to any timestamp without recomputation).

```
X_train, X_test, y_train, y_test = model.chronological_train_test_split(X, y)
pipe_with = model.build_pipeline(model.lightgbm_estimator())
pipe_with.fit(X_train, y_train)
scores_with = model.regression_scores(y_test, pipe_with.predict(X_test))

X_without, y_without = features.split_features_target(frame)
X_train_wo, X_test_wo, y_train_wo, y_test_wo = model.chronological_train_test_split(X_without
pipe_without = model.build_pipeline(model.lightgbm_estimator())
pipe_without.fit(X_train_wo, y_train_wo)
scores_without = model.regression_scores(y_test_wo, pipe_without.predict(X_test_wo))

print("WITH seasonal_24h:   ", {k: round(v, 4) for k, v in scores_with.items()})
print("WITHOUT seasonal_24h:", {k: round(v, 4) for k, v in scores_without.items()})
```

```
WITH seasonal_24h:    {'mae': 239.7615, 'rmse': 341.1845, 'mape': 0.0405}
WITHOUT seasonal_24h: {'mae': 239.278, 'rmse': 342.4446, 'mape': 0.0405}
```

An honest negative result: adding the seasonal-decomposition feature makes essentially no difference (identical MAPE to 4 decimal places). LightGBM's tree splits likely already extract everything useful from the cyclical `hour_sin`/`hour_cos` encoding plus the lag/rolling features — worth reporting the idea that didn't help, not only the ones that did.

5 Methodology

5.1 Why the train/test split is chronological, not shuffled

Lag and rolling-window features are built from each row's own recent past, so a random shuffle would leak near-future information into training rows sitting next to shuffled-in test rows. The split is by position along the time-sorted index (last 15% of hours as test), and cross-validation uses `TimeSeriesSplit`, never `KFold` — run on the **training period only**, so cross-validation and the final test are two independent measurements.

5.2 Two baselines, measured before trusting any model

What each model is actually asked to do matters here. Scored on `X_test`, every model makes a **1-hour-ahead** forecast: its `rolling_*_24h` features already include the hour just before the one being predicted. The honest no-model bar for that job is **persistence** — "demand will be what it was an hour ago". The "same hour last week" **seasonal naive** baseline is the fair bar for forecasts further ahead, where the previous hour isn't known yet.

```
baselines = pd.DataFrame({
    "Persistence (previous hour)": model.persistence_baseline(frame, y_test_wo),
    "Seasonal naive (same hour last week)": model.naive_seasonal_baseline(X_test_wo, y_test_
}).T
baseline_scores = baselines.loc["Seasonal naive (same hour last week)"]
persistence_scores = baselines.loc["Persistence (previous hour)"]
baselines.round(4)
```

	mae	rmse	mape
Persistence (previous hour)	244.4572	313.8748	0.0394
Seasonal naive (same hour last week)	442.9998	645.5151	0.0744

On the test period, "same hour last week" is off by **7.4%** on average (MAPE), but simply repeating the previous hour is off by only **3.9%**. Persistence is the bar the 1-hour-ahead models have to clear.

5.3 Model comparison, via TimeSeriesSplit on the training period

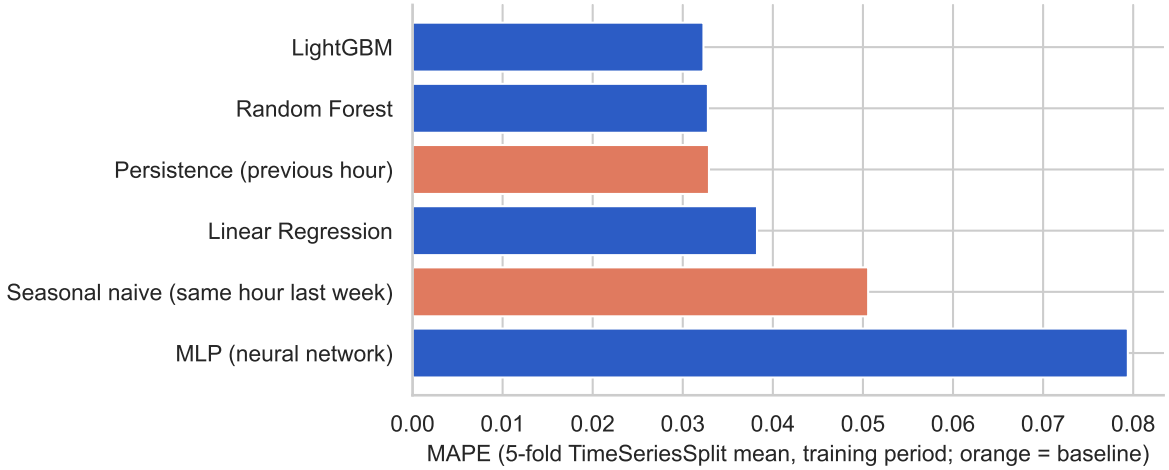
```
X_train_cv, y_train_cv = X_train_wo, y_train_wo
for i, (tr, va) in enumerate(model.time_series_folds(X_train_cv, cv=5), start=1):
    print(f"fold {i}: train {X_train_cv.index[tr[0]]:%Y-%m-%d} to {X_train_cv.index[tr[-1]]:%
          f"validate {X_train_cv.index[va[0]]:%Y-%m-%d} to {X_train_cv.index[va[-1]]:%Y-%m-%d
print(f"test period starts {X_test_wo.index.min()}")

cv_results = []
for name, factory in model.MODEL_FACTORIES.items():
    scores = model.cross_validate_pipeline(X_train_cv, y_train_cv, estimator=factory(), cv=5)
    cv_results.append({"model": name, **scores})
for name, scores in model.cross_validate_baselines(frame, X_train_cv, y_train_cv, cv=5).items():
    cv_results.append({"model": name, **scores})
cv_df = pd.DataFrame(cv_results).sort_values("mape_mean")
cv_df[["model", "mae_mean", "rmse_mean", "mape_mean", "mape_std"]]
```

```
fold 1: train 2019-01-08 to 2020-01-14, validate 2020-01-14 to 2021-01-20
fold 2: train 2019-01-08 to 2021-01-20, validate 2021-01-20 to 2022-01-26
fold 3: train 2019-01-08 to 2022-01-26, validate 2022-01-26 to 2023-02-02
fold 4: train 2019-01-08 to 2023-02-02, validate 2023-02-02 to 2024-02-08
fold 5: train 2019-01-08 to 2024-02-08, validate 2024-02-08 to 2025-02-14
test period starts 2025-02-14 13:00:00
```

	model	mae_mean	rmse_mean	mape_mean	mape_std
2	LightGBM	200.459415	277.595921	0.032297	0.008903
1	Random Forest	202.482090	299.766684	0.032789	0.006650
4	Persistence (previous hour)	211.646465	286.534196	0.032908	0.002010
0	Linear Regression	243.715006	325.570588	0.038240	0.005873
5	Seasonal naive (same hour last week)	322.120933	474.259811	0.050579	0.008155
3	MLP (neural network)	515.288671	647.726418	0.079406	0.090823

```
cv_idx = cv_df.set_index("model")
is_baseline = cv_df["model"].isin(baselines.index)
fig, ax = plt.subplots(figsize=(6.5, 3.4))
ax.barh(cv_df["model"], cv_df["mape_mean"], color=[PALETTE[3] if b else PALETTE[0] for b in i
ax.set_xlabel("MAPE (5-fold TimeSeriesSplit mean, training period; orange = baseline)")
plt.gca().invert_yaxis()
plt.show()
```



LightGBM is the best learned model (3.23% mean MAPE), ahead of Random Forest (3.28%) and Linear Regression (3.82%), and every model except the neural network clears the seasonal naive baseline (5.06%) comfortably. `MLP (neural network)` — a lightweight feedforward net (`sklearn.neural_network.MLPRegressor`, zero new dependencies) included specifically to test whether a neural approach earns its extra complexity — doesn't: it finishes last (7.94%) with by far the largest fold-to-fold spread (`mape_std` 9.08%).

The more sobering comparison is with **persistence** (3.29%): at one hour ahead, the best model is only marginally better than repeating the previous hour on these folds.

6 Final chronological holdout

```
holdout_rows = {}
for name, factory in model.MODEL_FACTORIES.items():
    pipe = model.train_pipeline(X_train_wo, y_train_wo, estimator=factory())
    holdout_rows[name] = model.regression_scores(y_test_wo, pipe.predict(X_test_wo))
    if name == "LightGBM":
        final_pipeline = pipe
holdout = pd.concat([pd.DataFrame(holdout_rows).T, baselines]).sort_values("mape")
test_preds = final_pipeline.predict(X_test_wo)
holdout.round(4)
```

	mae	rmse	mape
Persistence (previous hour)	244.4572	313.8748	0.0394
LightGBM	239.2780	342.4446	0.0405

	mae	rmse	mape
MLP (neural network)	258.8510	362.8145	0.0442
Random Forest	272.4969	396.5923	0.0466
Linear Regression	291.4789	402.7295	0.0485
Seasonal naive (same hour last week)	442.9998	645.5151	0.0744

On the untouched test period (February 2025 to March 2026), LightGBM is again the best of the four learned models (MAPE 4.05%, MAE 239 MW), and every model beats the seasonal naive baseline by a wide margin. But **it does not beat persistence one hour ahead**: repeating the previous hour scores 3.94% MAPE and 314 MW RMSE against LightGBM's 4.05% and 342 MW, with LightGBM ahead only on MAE (239 vs 244 MW). For a 1-hour-ahead forecast, the honest summary is a tie with the simplest possible rule. Demand changes slowly from one hour to the next, and the models only see the previous hour diluted inside a 24-hour rolling mean.

One caveat cuts the other way. Interpolated hours (the outage gaps and the removed fabricated stretches) are straight lines, which are trivially easy for persistence and hard for the model, so scoring only the real, non-interpolated test hours is a useful check:

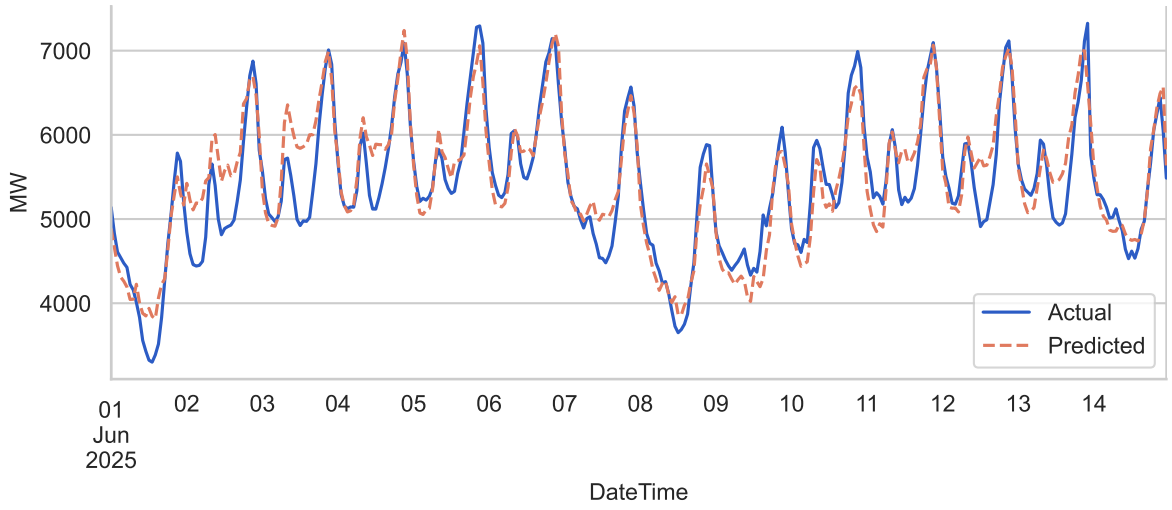
```
filled = ~y_test_wo.index.isin(df[config.DATETIME_COL]) | y_test_wo.index.isin(flagged.index
real = ~filled
pd.DataFrame({
    "LightGBM": model.regression_scores(y_test_wo[real], test_preds[real]),
    "Persistence (previous hour)": model.persistence_baseline(frame, y_test_wo[real]),
    "Seasonal naive (same hour last week)": model.naive_seasonal_baseline(X_test_wo[real], y
}).T.round(4).assign(hours=int(real.sum()))
```

	mae	rmse	mape	hours
LightGBM	231.6011	325.9478	0.0395	9243
Persistence (previous hour)	247.1447	316.0689	0.0399	9243
Seasonal naive (same hour last week)	435.1844	633.8472	0.0735	9243

On the real hours alone LightGBM edges ahead on MAPE and MAE but stays behind on RMSE — the verdict for one hour ahead is still a tie.

```
fig, ax = plt.subplots(figsize=(9, 3.2))
sample_start, sample_end = "2025-06-01", "2025-06-14"
y_test_wo.loc[sample_start:sample_end].plot(ax=ax, label="Actual", color=PALETTE[0])
pd.Series(test_preds, index=y_test_wo.index).loc[sample_start:sample_end].plot(ax=ax, label="
```

```
ax.set_ylabel("MW")
ax.legend()
plt.show()
```



7 Forecasting 24 hours ahead: where the model earns its keep

A grid operator needs the next day, not just the next hour. For hour *t+2* and later the previous hour isn't known yet, so persistence can only repeat the **last known** value, while the model forecasts recursively (`model.recursive_forecast`: predict one hour, feed the prediction back in, up to 24 hours; within that cap `lag_24h`, `lag_168h` and all generation-mix features stay real data). For a fair comparison, the LightGBM pipeline below is trained **only on the training period** (never the saved all-data model), and forecasts 24 hours from an anchor every 23 hours across the whole test period (so anchors cycle through every hour of the day), scored per step ahead against "repeat the last known value" and "same hour last week".

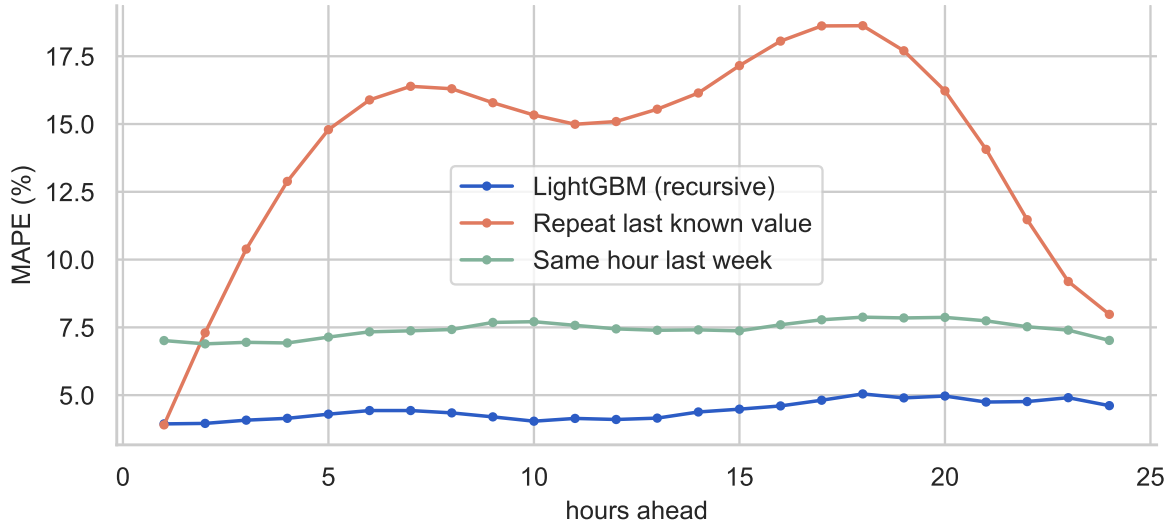
```
from energy_demand_forecasting import config

candidates = y_test_wo.index[: -(config.MAX_FORECAST_HORIZON_HOURS + 1)]
anchors = list(candidates[::23])
horizon = model.forecast_error_by_horizon(final_pipeline, frame, anchors).set_index("steps_al
horizon_labels = {"mape": "LightGBM (recursive)", "persistence_mape": "Repeat last known valu
                  "seasonal_naive_mape": "Same hour last week"}

fig, ax = plt.subplots(figsize=(8, 3.4))
for col, color in zip(horizon_labels, [PALETTE[0], PALETTE[3], PALETTE[4]]):
```

```
    (horizon[col] * 100).plot(ax=ax, marker=".", color=color, label=horizon_labels[col])
ax.set_xlabel("hours ahead")
ax.set_ylabel("MAPE (%)")
ax.legend()
plt.show()

blocks = {"1 h": slice(1, 1), "2-6 h": slice(2, 6), "7-12 h": slice(7, 12),
          "13-18 h": slice(13, 18), "19-24 h": slice(19, 24), "all 24 h": slice(1, 24)}
horizon_summary = pd.DataFrame({k: horizon.loc[v, list(horizon_labels)].mean() for k, v in bl
print(f"{len(anchors)} anchors, {horizon['n_anchors'].iloc[0]} scored at every step")
(horizon_summary * 100).round(2)
```




```
410 anchors, 410 scored at every step
```

	1 h	2-6 h	7-12 h	13-18 h	19-24 h	all 24 h
LightGBM (recursive)	3.94	4.18	4.21	4.58	4.82	4.44
Repeat last known value	3.90	12.25	15.65	17.36	12.77	14.16
Same hour last week	7.01	7.05	7.53	7.57	7.57	7.43

This is where the model is worth having. At step 1 it ties persistence (3.94% vs 3.90%), as in the holdout above. From step 2 on, repeating the last known value falls apart — it cannot follow the daily cycle, peaking at 18.6% and averaging 14.2% over the 24 hours — and "same hour last week" stays around 7.4%. The recursive forecast degrades only gently, from 3.9% one

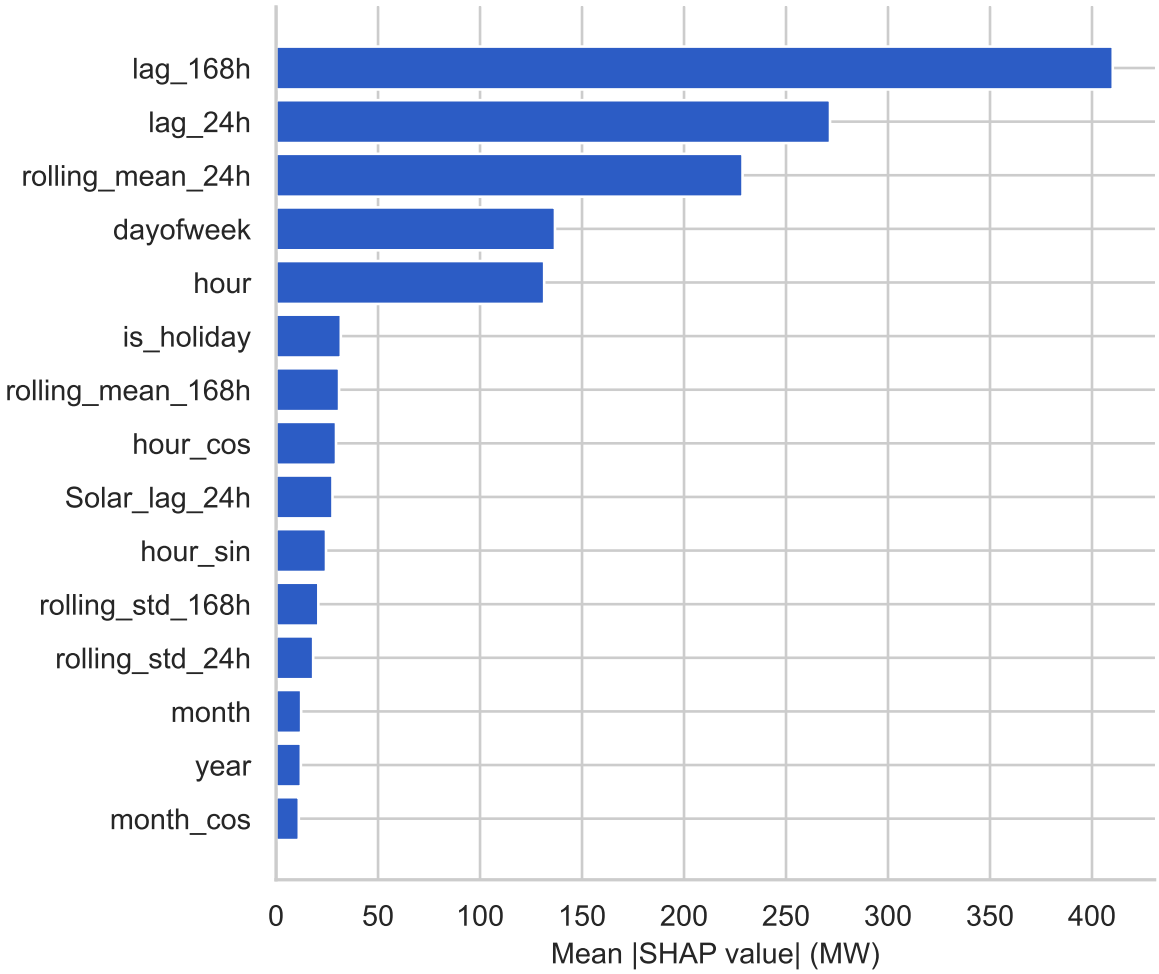
hour ahead to 4.8% on average for 19-24 hours ahead, and averages 4.44% over the full day. Over a 24-hour horizon, the better of the two baselines (same hour last week) makes about 1.7 times the model's error.

8 Model interpretability

```
from energy_demand_forecasting.interpretability import compute_shap_values, top_shap_features

explanation, X_transformed = compute_shap_values(final_pipeline, X_without, max_samples=500)
top = top_shap_features(explanation, top_n=15).sort_values("mean_abs_shap")

fig, ax = plt.subplots(figsize=(6, 6))
ax.barh(top["feature"], top["mean_abs_shap"], color=PALETTE[0])
ax.set_xlabel("Mean |SHAP value| (MW)")
plt.show()
```



`lag_168h` outranks `lag_24h` here, matching the ACF finding above where the weekly echo was slightly stronger. The genuine surprise: **`Solar_lag_24h` lands in the top 10**, just behind `is_holiday` and ahead of every other generation-mix feature. I didn't expect yesterday's solar output to say much about today's demand directly — my best guess is that it's acting as an indirect proxy for season and weather, not that solar generation itself drives consumption. I'd want more than a SHAP ranking alone before treating this as more than a proxy effect.

9 Discussion: what this project can't do, and what a Vietnam version would need

- **This dataset is live and not fully reproducible.** Every number above reflects one snapshot; a fresh download will add rows without changing historical ones, but the exact row counts and test-set contents quoted here are tied to this specific download.

- **The 2024-2025 outage gaps are patched with a rough approximation.** An 83-hour linear interpolation can't recover real demand swings for that stretch — evaluation numbers touching that period are measured on partly reconstructed, not fully real, data. The same goes for the fabricated 2025-2026 stretches the outlier rule removes: they are interpolated too, and milder copies that trip neither trigger remain.
- **One hour ahead, the model is no better than persistence.** The value it adds is over a multi-hour horizon. A model aimed squarely at the next hour would at least need the previous hour as its own feature (`lag_1h`), at the cost of faster error compounding when used recursively.
- **No weather data** — the sharp winter-heating seasonality is only seen through the calendar, never an actual temperature reading.
- **Different climate and market structure than Vietnam.** Romania's winter-heating-dominant demand shape is close to the opposite of Vietnam's likely summer-cooling-dominant shape. The *methodology* transfers directly; the specific calendar, climate-driven seasonality, and holiday structure would not.
- **The renewable-share angle connects most directly to Vietnam's own situation**: Vietnam's grid is going through its own fast solar and wind buildout, with well-documented grid-stability and curtailment challenges. A genuine Vietnam version of this analysis would need Vietnam's own hourly generation-mix data, which I could not find in public, granular form.

10 Conclusion

Switching from a frozen US demand series to this live Romanian consumption-and-production dataset paid off in ways beyond just having more columns: it forced a genuine leakage-trap decision (lagging every generation-mix feature), surfaced a real, previously-unseen data-quality problems (multi-hour 2024-2025 outage gaps, structurally different from the DST issue, and fabricated scaled-copy readings that had produced the dataset's apparent 12,000 MW "peak"), and gave two honest negative/surprising results worth reporting on their own terms — the seasonal-decomposition feature added nothing once properly leakage-checked, and yesterday's solar output ranks surprisingly high in the final model's own explanation of itself, for reasons I can guess at but haven't proven. LightGBM's win over both classical regression and a neural-network baseline held up on this dataset exactly as it did on the original US version — a second, independent piece of evidence that the highest common instinct ("try a neural net") isn't automatically the right one for a tabular, well-feature-engineered forecasting problem like this. The most important correction, though, came from adding the right baseline: one hour ahead, simply repeating the previous hour is as good as any model here, so the project's real claim is a narrower one — the model's value is in forecasting a whole day ahead, where it keeps its error at roughly 4-5% while the no-model alternatives sit at 7% or far worse.